❖

TECHNICAL REPORT · STACK & ARCHITECTURE

How the *D&D Companion* is built — and why

A mobile-first, offline-capable companion for D&D 5e: a browsable reference dump, player-owned characters, a customizable dashboard, and a live multiplayer session lobby — built to run from a phone with no server to babysit.

Type Installable PWA **Hosting** GitHub Pages + Firebase **Editions** 2014 & 2024 (5.5e)

374	219	21	9
UNIT TESTS	SOURCE FILES	DASHBOARD WIDGETS	RUNTIME DEPS

The guiding constraint shaped every choice: this had to feel like a native app on a phone, keep working with no signal, and stay cheap and simple to host. That pushes toward a lean front-end stack, on-device storage, and a strict separation between the read-only rules content and the things a player actually owns.

01 Foundation

the language & build

React
v19
UI RUNTIME

A component model maps cleanly onto a screen full of small, repeating pieces — stat blocks, widgets, roster rows. Its render model keeps the UI a **pure function of character + layout state**, so a data change redraws the right things without manual wiring.

TypeScript
strict mode

The app is, underneath, a pile of rules math — ability modifiers, proficiency, spell save DCs, AC. Strict types **catch those mistakes at**

TYPE SYSTEM

compile time instead of at the table. It also makes the codebase legible coming from a strongly-typed C#/Unity background.

Vite
v8
DEV SERVER & BUNDLER

Near-instant hot reload while building, a fast production build, and **first-class plugins** for both Tailwind and the offline service worker — so the tooling stays out of the way.

02 Interface

what you touch

Tailwind CSS
v4
STYLING

Utility-first styling driven entirely by **CSS variables**. That's the trick behind the theming system: dark, parchment, and six palette themes are a token swap on `<html>`, not a restyle. Mobile-first breakpoints match the phone-first goal.

React Router
v7
NAVIGATION

Client-side routing for the app shell — bottom nav, the Player / DM mode switch, the compendium, and character sheets — all without full page loads, so it behaves like an installed app.

dnd-kit
v6
DRAG & DROP

Reordering dashboard widgets and the initiative tracker needs drag-and-drop that's **built for touch**, not just a mouse. dnd-kit handles pointer + touch with proper tap-vs-drag activation on small screens.

03 State & storage

what the app remembers

Zustand
v5
APP STATE

Small, boilerplate-free stores — one per concern (character, scene, session, settings). No provider ceremony; any component reads exactly the slice it needs, which keeps re-renders tight on a mobile device.

Dexie · IndexedDB
v4
ON-DEVICE DATABASE

Characters and the cached 5e reference dump live **in the browser's own database**. The app owns its data, loads instantly, and keeps working with no network — the core of "offline-capable." Dexie

makes IndexedDB usable instead of painful.

04 Offline & realtime

install once, play together

vite-plugin-pwa
v1.3
INSTALLABILITY

Turns the site into an **installable app** with a home-screen icon and a precaching service worker, so it opens and runs offline like something from a store — without an app store.

Firebase Realtime DB
v12
MULTIPLAYER BACKEND

The session lobby is the one feature that genuinely needs a live server. GitHub Pages is static-only, so Firebase provides **realtime sync + presence** (players appear on the DM's screen, HP updates live, sleep shows "away"). It dropped in behind the existing repository interface — a swap, not a rewrite.

05 Correctness

the math has to be right

Vitest
v4
UNIT TESTING

The rules engine — modifiers, proficiency, spell slots, class & subclass resources, level 1–20 progression — is pure and heavily tested. **374 tests** reconcile the app's math against the real reference data so a refactor can't quietly break a character.

ESLint + typescript-eslint
v10 / v8
STATIC ANALYSIS

The compiler is the first test; lint keeps the React hooks and render model honest. Together they're the gate every build passes before it ships.

◆ Why it's shaped this way

non-negotiable rules

The stack is only half the story; a handful of architectural rules keep the app extensible and the two kinds of data from ever tangling.

❖ Two data domains, never mixed

Read-only **reference** content (spells, monsters, classes) is kept strictly apart from read/write **user** data

(characters, layouts, DM state). They have separate stores and separate rules.

❖ References, not copies

A character stores a `name|source` id, never a copy of the spell or item, and resolves the real content at render time — so fixing the data never desyncs a saved sheet.

❖ A pure core

The `domain/` and `rendering/` layers contain **no React and no DOM** — just plain functions. That purity is exactly why the rules math is unit-testable in isolation.

❖ Swappable data sources

All persistence sits behind repository **interfaces**. Dexie backs them today; Firebase backs the lobby; a future sync backend would be an injection, not a rewrite. The multiplayer feature proved this out.

❖ A data-driven dashboard

The home screen is a list of widget instances mapped through a registry. A new widget type is a **registration**, never an edit to the engine — Open/Closed in practice. One generic resource-tracker covers every class resource.

❖ Rendering that degrades gracefully

The reference text uses a `{@tag}` markup that a handler registry expands into rich UI. Unknown tags **fall back to plain text** instead of throwing — the app never breaks on data it doesn't recognize.

THE THROUGH-LINE

Own your data, resolve references late, keep the core pure, and hide every backend behind an interface.

Those four ideas are why a single-player offline app could grow a live multiplayer lobby without disturbing anything that already worked — and why the next big feature should be able to do the same.

D&D Companion — technical report React 19 · TypeScript · Vite · Tailwind v4 · Zustand · Dexie · Firebase